Verification of Digital Systems (02207)

Linux Environment Setup

2026 SyoSil ApS ©

Contents

1 Introduction **3**

2 *Docker Desktop* setup Windows **4**
2.1 *WSL* installation 4
2.2 *Docker* Desktop: Installation 4
2.3 *Docker* Desktop: Enable applications interface inside the *Docker* container 4
2.4 *Docker* Desktop: Create and run *Docker* container 6

3 *Docker Desktop* setup MacOS **7**
3.1 *Docker* Desktop: Installation 7
3.2 *Docker* Desktop: Enable applications interface inside the *Docker* container 7
3.3 *Docker* Desktop: Create and run *Docker* container 8

4 *Docker* setup Linux **9**
4.1 *Docker* Engine Installation 9
4.2 *Docker* Engine: Enable applications interface inside the *Docker* container 9
4.3 *Docker* Engine: Create and run *Docker* container 9

5 Additional Setup for Visual Studio Code **10**

6 Gitlab setup **11**
6.1 Cloning repository 11

References **12**

A Appendices **13**
A.1 Linux terminal basics 13
A.2 Git 13
A.3 Troubleshooting 13
A.4 Low Memory in WSL2 on Windows 13
A.4.1 GUI is not showing in docker 13
A.5 Useful *Docker* commands 13

1 Introduction

This document describes how to set up the Linux environment using *Docker-Desktop*[1].

There are three versions of the installation guide avaliable:

1. A Windows version, see section 2
2. A MacOS version, see section 3
3. A Linux version, see section 4

Docker is a tool for running software in isolated environments called *containers*. The provided *Docker image* contains all necessary tools and is ready to use. The *Docker* setup consists of three main parts:

- The ***Docker* image** (`syosil-ubuntu-image`), contains all the prebuild binaries, configurations etc. for the tools. The image only has to be build once.
- The ***Docker* container** (`syosil-ubuntu-container`), is a specific instance running the image. Containers are isolated environments where an application runs without affecting the rest of the host system and without the host system impacting the application. The provided script to run the container creates a persistent container mounted on a *Docker* volume.
- The ***Docker* volume** (`<USERNAME>-docker-workdir`) provides persistent storage shared between the host and container. Even if the container is deleted, the volume will persist and a new container with the same volume can be created.

The *Docker* GUI provides an overview of all existing *images*, *containers*, and *volumes*.

Following this guide will install the `syosil-ubuntu-image`, create the `syosil-ubuntu-container`, and a *Docker* volume named `<USERNAME>-docker-workdir`. The volume is mounted to the home directory (`/home/<USERNAME>-docker`) in the container. Generally the container only needs to be created once and can be reused during the course. If the container is removed the volume will persist and when creating another container using the provided scripts the volume is mounted to the home directory again.

The following tools are installed in the *Docker* container:

- GTKWave: Wave viewer.
- Surfer: Wave viewer.
- Icarus Verilog: HDL compiler and simulator.
- Python version 3.12.0

HINT

Some commands and file paths in the following sections contain values in angle brackets (`<...>`). These are placeholders for values that will vary for each machine.

For example

- `C:\Users\<WINDOWS-USERNAME>` denotes the specific windows file path to the users home directory, e.g. `C:\Users\syosil`

2 *Docker Desktop* setup Windows

This section describes the steps for:

- installing ***WSL***;
- installing *Docker Desktop*;
- installing the requirements to access to the GUI of an application running in the *Docker* container;
- creating and running the *Docker* container.

2.1 *WSL* installation

Docker Desktop uses *WSL2* for virtualization on Windows. You can check if *WSL2* is installed by running:

```
C:\Users\<WINDOWS-USERNAME>> wsl -l -v
```

If you can see an installed distribution with '2' in the version field (see example below) all is good, otherwise follow the steps in `https://learn.microsoft.com/en-us/windows/wsl/install` to install *WSL*.

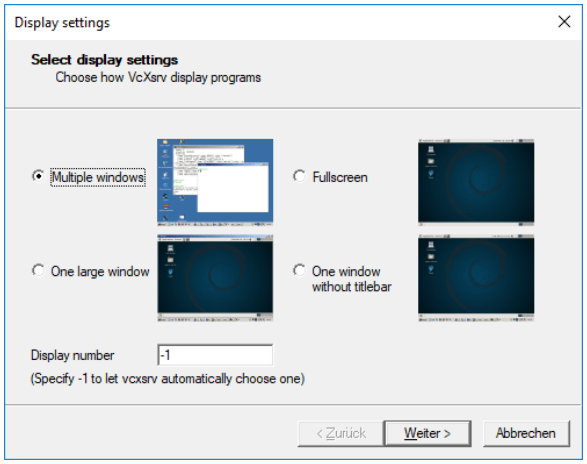
```
C:\Users\syosil> wsl -l -v
  NAME              STATE           VERSION
* Ubuntu            Stopped         2
  docker-desktop    Stopped         2
```

2.2 *Docker* Desktop: Installation

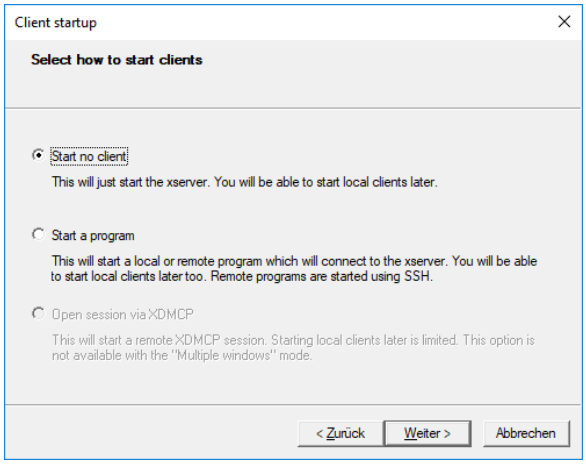
1. Download *Docker Desktop* from the official website:
   `https://docs.docker.com/desktop/install/windows-install/`
2. Open the downloaded file and run the installer.
3. Start *Docker* Desktop and go though the initial setup.

2.3 *Docker* Desktop: Enable applications interface inside the *Docker* container

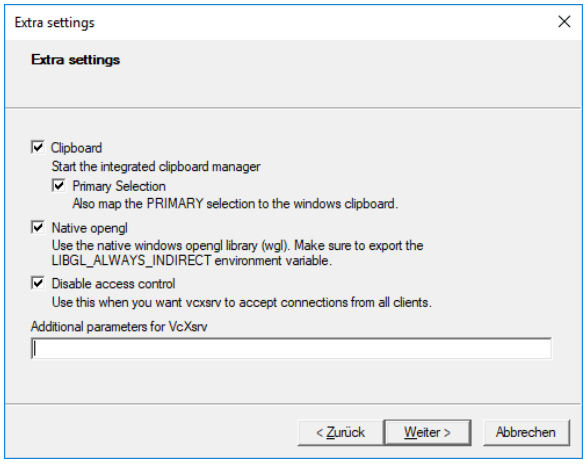
1. Download ***VcXsrv Windows X Server*** from:
   `https://sourceforge.net/projects/vcxsrv/`
2. Open the file downloaded and run the installer.
3. After successful installation, open **Xlaunch** from the start menu, and follow the configuration in Figure 2.1.
   - Make sure to *Disable access control* (see Figure 2.13).
   - **Save the configuration file before you click finish!** (see Figure 2.14)



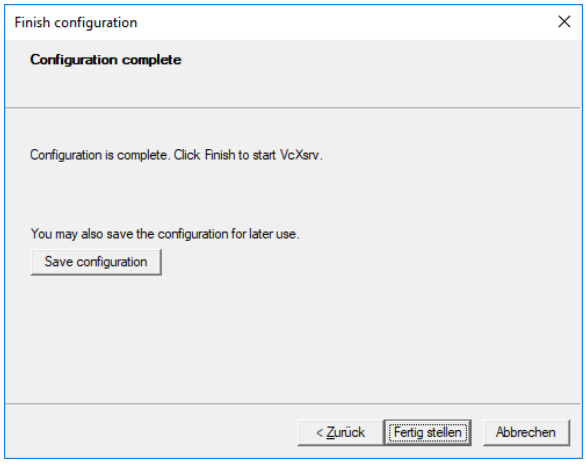
(1) Select "Multiple windows" and click "Next".



(2) Select "Start no client" and click "Next".



(3) Select all options and click "Next".



(4) Click on "Save configuration" and then click "Next".

Fig. 2.1: Configuration steps for **XLaunch**

2.4 *Docker* Desktop: Create and run *Docker* container

1. Make sure *Docker* Desktop is running.
2. Download `verification_env_setup.zip` from the course page and unpack it.
3. Open **Powershell** in the unpacked `verification_env_setup`-folder and run the following command to build the *Docker container*.

```
C:\Users\<WINDOWS-USERNAME>\verification_env_setup> .\setup_command_build_win.bat
```

4. Then, run the following command to create the *Docker container*.

```
C:\Users\<WINDOWS-USERNAME>\verification_env_setup> .\setup_command_run_win.bat
```

5. To stop the container run `exit` in the *Docker* shell. To tart the container again run:

```
C:\Users\<WINDOWS-USERNAME>\verification_env_setup> docker start --attach --interactive
syosil-ubuntu-container
```

6. The container is set up and all the software installed. Continue with section 5.

3 *Docker Desktop* setup MacOS

This section describes the steps for:

- installing *Docker Desktop*;
- installing the requirements to access to the GUI of an application running in the *Docker* container;
- creating and running the *Docker* container.

3.1 *Docker* Desktop: Installation

1. Download *Docker Desktop* from the official website:
   `https://docs.docker.com/desktop/install/mac-install/`
2. Open the downloaded file and run the installer.
3. Start *Docker* Desktop and go though the initial setup.

3.2 *Docker* Desktop: Enable applications interface inside the *Docker* container

1. Download ***XQuarz*** from, `https://www.xquartz.org/`.
2. Open the file downloaded and run the installer.
3. After successful installation, launch ***XQuarx*** and open the **App Preferences**.
   - Navigate to the security tab.
   - Check the *Allow connections from network clients* box. (See Figure 3.1)
   - Restart ***XQuarz***.
4. Open the terminal and run:

```
<username>@<machine-name> $ xhost +localhost
<username>@<machine-name> $ xhost +127.0.0.1
```

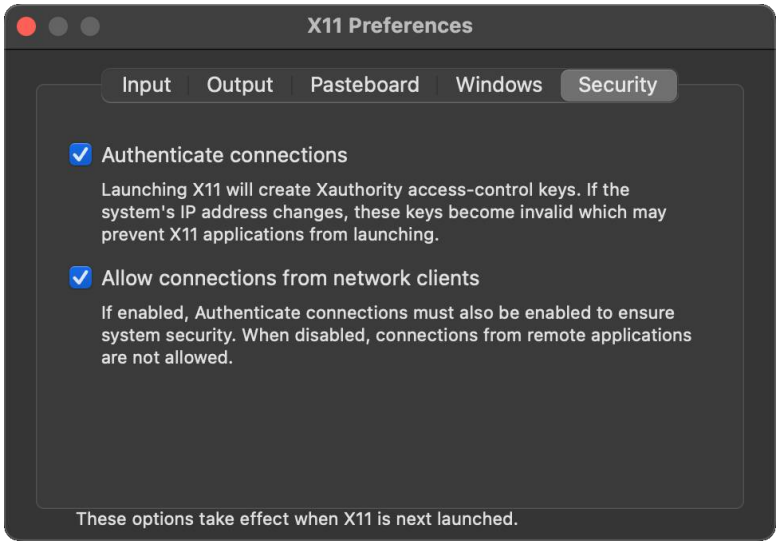


Fig. 3.1: Configuration settings for **XQuartz**.

3.3 *Docker* Desktop: Create and run *Docker* container

1. Make sure *Docker* Desktop is running.
2. Download `verification_env_setup.zip` from the course page and unpack it.
3. Open the **Terminal** and navigate to the unpacked `verification_env_setup`-folder.
4. Execute the `setup_command_build.sh` by sourcing it:

```
<username>@<machine-name> ~/verification_env_setup $ source setup_command_build.sh
```

5. Then, execute `setup_command_run.sh`:

```
<username>@<machine-name> ~/verification_env_setup $ source setup_command_run.sh
```

6. To stop the container run `exit` in the *Docker* shell. To start the container again run:

```
<username>@<machine-name> ~/verification_env_setup $ docker start --attach --interactive
syosil-ubuntu-container
```

7. The container is set up and all the software installed. Continue with section 5.

4 *Docker* setup Linux

This section describes the steps for:

- installing *Docker*;
- creating and running the *Docker* container.

4.1 *Docker* Engine Installation

1. Follow the instructions for installing the *Docker* engine for your distribution from the official website: `https://docs.docker.com/engine/install/`

4.2 *Docker* Engine: Enable applications interface inside the *Docker* container

This step is not necessary on Linux as X11 is supported natively.

4.3 *Docker* Engine: Create and run *Docker* container

1. Download `verification_env_setup.zip` from the course page and unpack it.
2. Open the **Terminal** and navigate to the unpacked `verification_env_setup`-folder.
3. Execute the `setup_command_build.sh` by sourcing it:

```
<username>@<machine-name> ~/verification_env_setup $ source setup_command_build.sh
```

4. Then, execute `setup_command_run.sh`:

```
<username>@<machine-name> ~/verification_env_setup $ source setup_command_run.sh
```

5. To stop the container run `exit` in the *Docker* shell. To start the container again run:

```
<username>@<machine-name> ~/verification_env_setup $ docker start --attach --interactive
syosil-ubuntu-container
```

6. The container is set up and all the software installed. Continue with section 5.

5 Additional Setup for Visual Studio Code

The *Docker* container has three different editors installed:

- Vim
- Emacs
- Nano

Alternatively Visual Studio Code can also be used:

- The *Docker* container can be accessed directly from Visual Studio Code. See `https://code.visualstudio.com/docs/devcontainers/attach-container` for more information.
- The Surfer waveform view can also be installed as a Visual Studio Code extension. See `https://marketplace.visualstudio.com/items?itemName=surfer-project.surfer` for more information.

6 Gitlab setup

The project files and code for this course will be shared via a repository on the DTU GitLab (`https://gitlab.gbar.dtu.dk`). Two repositories will be provided:

1. A working repository, containing the necessary code to do the exercises.
2. A solutions repository, containing the solutions to the exercises.

The content of these repositories will be updated weekly. The solutions to the exercises will be made available in the following week. Each weeks state of the repositories is preserved on a dedicated branch e.g. `Lecture-1`. The `main` branch will always contain the newest changes see Figure 6.1.

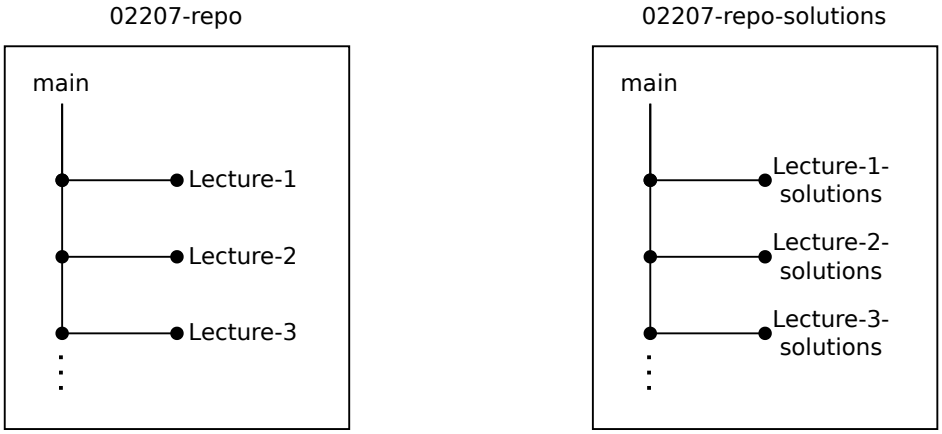


Fig. 6.1: Branches of the course repositories.

The repositories are read-only and any work should be carried out locally or on a private repository on GitLab.

6.1 Cloning repository

To clone the working repository, a ssh key needs to be created in the *Docker* container and added to the authorized ssh-keys in GitLab. To create a new ssh-key and add it to your GitLab accout follow the steps in the GitLab documentation: `https://docs.gitlab.com/user/ssh/#generate-an-ssh-key-pair`

Then clone the repository by running:

```
<username>@<machine-name> ~/ $ git clone <repository-url>
```

If `ssh` is not able to find your ssh key, it is most likely due to the the ssh-key file having a different name than the standard name (e.g. `~/.ssh/id_rsa`/`~/.ssh/id_ed25519`). Assuming the ssh key is placed in the file `~/.ssh/<private key file name>`, add the following to `~/.ssh/config`:

```
Host gitlab.gbar.dtu.dk
   HostName gitlab.gbar.dtu.dk
   User git
   IdentityFile ~/.ssh/<private key file name>
```

For more information see: `https://docs.gitlab.com/topics/git/clone/`

References

[1] Docker, “Overview of Docker Desktop.” [Online]. Available: https://docs.docker.com/desktop/

A Appendices

A.1 Linux terminal basics

If you are not familiar with linux terminal basics the following links provide some recommended reading.

- A basic overview of the most important Linux commands can be found here: `https://www.geeksforgeeks.org/linux-unix/basic-linux-commands/`
- For a more comprehensive beginners guide see: `https://www.ubuntu.com/tutorials/command-line-for-beginners`

A.2 Git

If you are not familiar with git the following links provide some recommended reading.

- Minimal guide: `https://rogerdudler.github.io/git-guide/`
- Basic command overview: `https://education.github.com/git-cheat-sheet-education.pdf`
- Comprehensive documentation: `https://git-scm.com/book/en/v2`
- Interactive learning: `https://ohmygit.org/`

A.3 Troubleshooting

A.4 Low Memory in WSL2 on Windows

Some Windows systems might run out of memory during the image building process. It might help to tweak the memory and swap file size in the *WSL*-Settings. The *WSL*-Settings app can be found in the Start menu.

A.4.1 GUI is not showing in docker

If no GUI shows up when starting *gtkwave* or the *pyucis-viewer* from the docker container, consider the following troubleshooting steps:

- Check if **XLaunch**/**XQuartz** is running.
- Check if **XLaunch**/**XQuartz** has been configured according to subsection 2.3/subsection 3.2
  - For Windows especially if access control has been disable in the **XLaunch** setting.
  - For Mac especially if *Allow connections from network clients* in the **XQuarz** settings is set.
- Check if `echo $DISPLAY` is set to `host.docker.internal:0.0`.

A.5 Useful *Docker* commands

```
<WSL-USERNAME>-docker@docker-container:~/$ docker --help       # Docker help

<WSL-USERNAME>-docker@docker-container:~/$ sudo docker images  # List Docker images
<WSL-USERNAME>-docker@docker-container:~/$ sudo docker ps -a   # List all containers

<WSL-USERNAME>-docker@docker-container:~/$ docker start --attach --interactive <container_name>  # Start
a container and attach to it
<WSL-USERNAME>-docker@docker-container:~/$ docker attach <container_name>          # Attach to the
current terminal a running container
<WSL-USERNAME>-docker@docker-container:~/$ docker stop <container_name>            # Stop a running
container
```

```
<WSL-USERNAME>-docker@docker-container:~/$ docker rmi <IMAGE ID>       # Remove image
<WSL-USERNAME>-docker@docker-container:~/$ docker rm <CONTAINER ID>    # Remove container
```
